DormoDaTE

1. Analisi concettuale

1.1. Diagramma Uml

1.1.1. Specifiche dei tipi di dato

1.1.2. Specifiche delle classi

Relazioni ausiliarie

giorno_di_data(d: Data, g: GiornoSettimana)

data_di_DataOra(do: DataOra, d: Data)

Operazioni ausiliarie

sovrapposti_DataOra(

non_sovrapposti_Data(

data_chiusa_da_periodoChiusura(p: PeriodoChiusura, d: Data) : Booleano

data_chiusa_da_chiusuraStraordinaria(cs: ChiusuraStraordinaria, d: Data) : Booleano

Vincoli esterni

[Utente.Offre_alloggio_solo_Ospittante]

[Utente.Prenotazioni_non_sovrapposte]

[Utente.NuovaPrenotazione_solo_dopo_Feedback_PrecedentePrenotazione]

[SceltaPosto.coerenza_Prenotazione_Visitatore_Letto_collegati]

[Alloggio.NumOspiti_minore_uguale_PostiDisponibili]

[Letto.Capienza_minore_uguale_MaxCapienza]

[Letto.Num_SceltaPosto_minore_uguale_Capienza]

[Letto.PrenAccettate_non_sovrapposte_se_letto_non_condivisibile]

[Prenotazione.DataInizio_minore_DataFIne]

[Prenotazione.Letti_assegnate_appartengono_allo_stesso_Alloggio]

[PeriodoChiusura.GiornoSettimana_diversi]

[PeriodoChiusura.Inizio_minore_Fine]

[PeriodoChiusura.Opzionale_GiornoSettimana]

[PeriodoChiusura.Periodi_non_sovrapposti]

[ChiusuraStraordinaria.Inizio_minore_Fine]

[ChiusuraStraordinaria.chiusure_non_sovrapposte]

Operazioni di classe Utente

eta(): Intero > 0

Operazioni di classe Alloggio

Operazioni ausiliarie

alloggio_chiuso_in_data(d: Data): Booleano

alloggio_chiuso_in_periodo(d_in: Data, d_fin: Data): Booleano

alloggio_posti_totali(): Intero > 0

alloggio_posti_prenotati_in_periodo(d_in: DataOra, d_fin: DataOra): Intero >= 0

Operazioni di classe

alloggio_posti_liberi_in_periodo(d_in: DataOra, d_fin: DataOra): Intero >= 0

Operazioni di classe Letto

Operazioni ausiliarie

posti_in_letto_prenotati_in_periodo(d_in: DataOra, d_fin: DataOra): Intero >= 0

Operazioni di classe

posti_in_letto_liberi_in_periodo(d_in: DataOra, d_fin: DataOra): Intero >= 0

letto_libero_in_periodo(d_in: DataOra, d_fin: DataOra): Booleano

1.2. Diagramma Use-Case

1.2.1. Operazioni di use-case Registrazione al portale

registrazione_utente(

1.2.2. Operazioni di use-case Gestione alloggi

registrazione_alloggio(
creazione_camera_in alloggio(al: Alloggio, nom: Stringa) : Camera
creazione_unita_letto(
creazione_periodo_chiusura(
eliminazione_periodo_chiusura(
creazione_chiusuraStraordinaria(
eliminazione_chiusura_straordinaria(

1.2.3. Operazione di use-case Strumenti ricerca alloggio

ricerca_ospitanti_con_posti_liberi_in_periodo_in_citta(

1.2.4. Operazione di use-case Strumenti prenotazioni

creazione_prenotazione(

1.2.5. Operazione di use-case Gestione prenotazioni

ospitante_accetta_prenotazione(pren: Prenotazione) : PrenAccettata
ospitante_rifiuta_prenotazione(pren: Prenotazione, motiv: Stringa) : PrenRifiutata

1.2.6. Operazione di use-case Strumenti valutazione

viaggiatore_valuta_ospitante(
ospitante_valuta_visitatore(

1.2.7. Operazione di use-case Gestione Ubicazione

creazione_tipo_letto(
creazione_tipo_stazione(nom: Stringa) : TipoStazione
creazione_tipo_alloggio(nom: Stringa) : TipoAlloggio
creazione_citta(nom: Stringa, reg: Regione) : Citta
creazione_regione(nom: Stringa, naz: Nazione) : Regione
creazione_nazione(nom: Stringa) : Nazione

2. Ristrutturazione per Basi Dati

2.1. Diagramma UML ristrutturato per bd

2.2. Specifica dei vincoli esterni

[Prenotazione.Disgiunzione]
[Prenotazione.organizzatore_isa_visitatore]

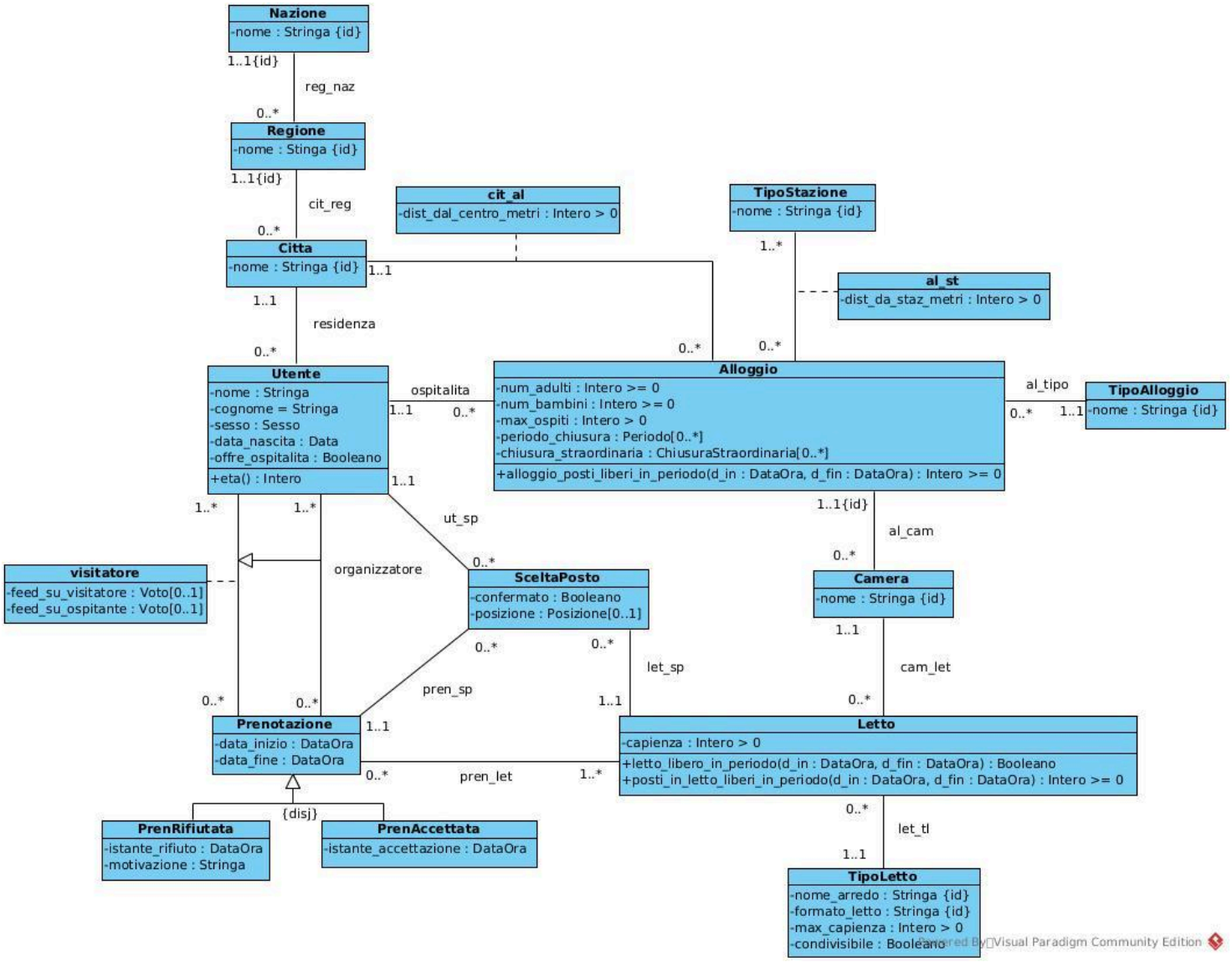
2.3. Traduzione diretta

2.3.1. Tipi di dato

2.3.2. Schema relazionale

1. Analisi concettuale

1.1. Diagramma Uml



Ho creato la classe SceltaPosto per dare la possibilita agli utenti di indicare esattamente quali letti vogliono prenotare. (Cosi si puo evitare gran casino quando nello stesso alloggio vengono a dormire piu gruppi di persone).
Ho aggiunto attributo chiusura_straordinaria in Alloggio per permettere all'ospitante di chiudere alloggio in caso di imprevisti senza toccare periodo_chiusura esistenti. La chiusura_srtaordinaria puo essere creata solo se in quel periodo non ci sono prenotazioni Accettate.

1.1.1. Specifiche dei tipi di dato

Tipo Sesso: {maschio, femmina}

Tipo GiornoSettimana: {lunedì, martedì, mercoledì, giovedì, venerdì, sabato, domenica}

Tipo PeriodoChiusura: Tipo record con campi
- data_inizio: Data
- data_fine: Data
- giorno_settimana: GiornoSettimana [0..7]

Tipo ChiusuraStraordinaria: Tipo record con campi
- data_inizio: Data

data_fine: Data
motivo: Stringa[0..1]

Tipo Voto: Tipo intervallo di interi tra [0..5]

Tipo Posizione: {sopra, sotto} #serve per il letto a castello

Tipo Preferenze: tipo record con campi
utente: Utente
letto: Letto
posiz: Posizione [0..1]

1.1.2. Specifiche delle classi

Relazioni ausiliarie

giorno_di_data(d: Data, g: GiornoSettimana)

$\forall\, d,\ g\ \ (\, giorno_di_data(d,\ g)\ \Rightarrow\ Data(d)\ \ \wedge\ \ GiornoSettimana(g)\,)$

$\forall\, d\ \ (\, Data(d)\ \Rightarrow \exists!\ g\ (GiornoSettimana(g)\ \wedge\ giorno_di_data(d,\ g)\,)$

→ [PeriodoChiusura.opzionale_GiornoSettimana]

data_di_DataOra(do: DataOra, d: Data)

$\forall\, do\ (\, DataOra(do)\ \Rightarrow\ \ \exists!\ d\ (Data(d)\ \ \wedge\ \ data_di_DataOra(do,\ d))$

→alloggio_posti_liberi_in_periodo(.)

Operazioni ausiliarie

sovrapposti_DataOra(
d_in1: DataOra,
d_fin1: DataOra,
d_in2: DataOra,
d_fin2: DataOra
): Boolean

precondizione:
- *d_in1 < d_fin1 e d_in2 < d_fin2*

postcondizione:
- operazione non fa side-effect
- *result* = True
  - se e solo se *d_in1 < d_fin2* **e** *d_in2 < d_fin1*

Pre:

$<\ (d_in_1,\ d_fin_1)\ \wedge\ \ <\ (d_in_2,\ d_fin_2)$

Post:

$$sovrapposti_DataOra(\, d_in_1,\, d_fin_1,\, d_in_2,\, d_fin_2,\, True)$$

$$\Leftrightarrow \quad <(d_in_1,\, d_fin_2) \;\wedge\; <(d_in_2,\, d_fin_1)$$

→[Utente.Prenotazioni_non_sovrapposti]

→[Letto.PrenAccettate_non_sovrapposte_se_letto_non_condivisibile]

→posti_in_letto_prenotati_in_periodo(.)

non_sovrapposti_Data(

d_in1: Data,
d_fin1: Data,
d_in2: Data,
d_fin2: Data
): Boolean

precondizione:

- *d_in1 < d_fin1 e d_in2 < d_fin2*

postcondizione:

- operazione non fa side-effect
- *result* = True
  - se e solo se *d_fin1* <= *d_in2* oppure *d_fin2* <= *d_in1*

Pre:

$$<(d_in_1,\, d_fin_1) \;\wedge\; <(d_in_2,\, d_fin_2)$$

Post:

$$non_sovrapposti_Data(\, d_in_1,\, d_fin_1,\, d_in_2,\, d_fin_2,\, True)$$

$$\Leftrightarrow \quad \leq(d_fin_1,\, d_in_2) \;\vee\; \leq(d_fin_2,\, d_in_1)$$

→[PeriodoChiusura.Periodi_non_sovrapposti]

→[ChiusuraStraordinaria.chiusure_non_sovrapposte]

→creazione_periodo_chiusura(.)

→creazione_chiusuraStraordinaria(.)

Tipo PeriodoChiusura: Tipo record con campi
data_inizio: Data
data_fine: Data
giorno_settimana: GiornoSettimana [0..7]

data_chiusa_da_periodoChiusura(p: PeriodoChiusura, d: Data) : Booleano

Precondizioni: nessuna

Postcondizioni:

- operazione non fa side-effect
- result = True se e solo se
  (*p.data_inizio* <= d < *p.data_fine*)
  AND
  (*p.giorno_settimana* non è valorizzato
  OR

d.giorno_settimana è in *p.giorno_settimana*)

Pre: nessuna
Post:

$$\forall\, p,\ d\ \ (PeriodoChiusura(p)\ \wedge\ Data(d)$$
$$\Rightarrow\ (\ data_chiusa_da_periodoChiusura(p,\ d,\ True)$$
$$\Leftrightarrow\ (\ (\ \leq (data_inizio(p),\ d)\ \ \wedge\ \ <(d,\ data_fine(p)\ )$$
$$\wedge\ \ (\ (\ \neg\exists\, g\ (GiornoSettimana(g)\ \wedge\ giorno_settimana(p,\ g)\ )$$
$$\vee$$
$$(\ \exists\, g\ \ (GiornoSettimana(g)$$
$$\wedge\ \ giorno_di_data(d,\ g)$$
$$\wedge\ \ giorno_settimana(p,\ g)\ )\ )\ \ )\ \ )\ )$$

→alloggio_chiuso_in_data(.)
→[PeriodoChiusura.opzionale_GiornoSettimana]

data_chiusa_da_chiusuraStraordinaria(cs: ChiusuraStraordinaria, d: Data) : Booleano

Precondizioni: nessuna
Postcondizioni:

- operazione non fa side-effect
- result = True se e solo se
  (cs.data_inizio <= d < cs.data_fine)

Pre: nessuna
Post:

$$\forall\, cs,\ d\ (\ ChiusuraStraordinaria(cs)\ \wedge\ Data(d)$$
$$\Rightarrow\ \ (\ data_chiusa_da_chiusuraStraordinaria(p,\ d,\ True)$$
$$\Leftrightarrow\ \ \leq (data_inizio(cs),\ d)\ \ \wedge\ \ <(d,\ data_fine(cs))\ )\ )$$

→alloggio_chiuso_in_data(.)

Vincoli esterni

[Utente.Offre_alloggio_solo_Ospittante]

Per ogni ut: Utente tale che ut.offre_ospitalita = True deve esistere al: Alloggio tale che (ut, al): ospitalita

$$\forall\, ut\ (\ Utente(ut)\ \ \wedge\ \ offre_ospitalita(ut,\ True)\ )\ \Rightarrow \exists\, al\ \ (\ Alloggio(al)\ \ \wedge\ \ ospitalita(al,\ ut)\ \ )$$

[Utente.Prenotazioni_non_sovrapposte]

Per ogni *ut: Utente* e per ogni *pren1, pren2: PrenAccettata* tali che

- se esistono i link
  - *(ut, pren1): visitatore* e *(ut, pren2): visitatore*
  - *pren1* ≠ pren2

deve essere vero che

- sovrapposti_DataOra(pren1.data_inizio,
  pren1.data_fine,
  pren2.data_inizio,

pren2.data_fine
) = False

$$\forall\, ut,\ pren_1,\ pren_2\ (\ Utente(ut)\ \wedge\ PrenAcettata(pren_1)\ \wedge\ PrenAcettata(pren_2)$$
$$\wedge\ visitatore(ut,\ pren_1)\ \wedge\ visitatore(ut,\ pren_2)$$
$$\wedge\ pren_1 \neq pren_2\ )$$
$$\Rightarrow\ sovrapposti_DataOra($$
$$data_inizio(pren_1),\ data_fine(pren_1),$$
$$data_inizio(pren_2),\ data_fine(pren_2),\ \ False)\ )$$

[Utente.NuovaPrenotazione_solo_dopo_Feedback_PrecedentePrenotazione]

Per ogni *ut: Utente* e per ogni *pren1: PrenAccettata,* tali che

- esiste (ut, pren1): visitatore e (ut, pren1).feed_su_ospitante non è valorizzato

deve essere vero che

- non essistere pren2: Prenotazione tale che
  - pren1≠ pren2
  - (ut, pren2): visitatore
  - pren2.data_inizio > pren1.data_fine

$$\forall\, ut,\ pren_1\ (\ Utente(ut)\ \wedge\ PrenAcettata(pren_1)$$
$$\wedge\ visitatore(ut,\ pren_1)$$
$$\wedge\ (\ \neg\exists\ voto\ \ (\ Voto(voto)$$
$$\wedge\ feed_su_ospitante(ut,\ pren_1,\ voto)\ )$$
$$\Rightarrow\ \neg\exists\ pren_2(\ Prenotazione(pren_2)$$
$$\wedge\ visitatore(ut,\ pren_2)$$
$$\wedge\ pren_1 \neq pren_2$$
$$\wedge\ \ \geq\ (\ data_inizio(pren_2),\ data_fine(pren_1)\ )$$

[SceltaPosto.coerenza_Prenotazione_Visitatore_Letto_collegati]

Per ogni ut: Utente, sp: SceltaPosto, pren: Prenotazione, let: Letto
tale che esistono

- (ut, sp): ut_sp, ut Utente
- (pren, sp):pren_sp
- (let, sp): let_sp

deve essere vero che

- esistono i link (ut, pren): visitatore e (pren, let): pren_let

$$\forall\, ut,\ pren,\ sp,\ let\ \ \ (\ Utente(ut)\ \wedge\ Prenotazione(pren)$$
$$\wedge\ SceltaPosto(sp)\ \wedge\ Letto(let)$$
$$\wedge\ ut_sp(ut,\ sp)$$
$$\wedge\ pren_sp(pren,\ sp)$$
$$\wedge\ let_sp(let,\ sp)$$

$\Rightarrow visitatore(pren, ut) \ \wedge pren_let(pren, let)\ )$

[Alloggio.NumOspiti_minore_uguale_PostiDisponibili]

Per ogni al: Alloggio deve essere vero che *al.max_ospiti* <= *al.alloggio_posti_totali()*

$\forall\, al \ \ (\, Alloggio(al)$

$\Rightarrow \ \leq (max_ospiti(al)\,,\ alloggio_posti_totali(al))\ )$ # $alloggio_posti_totali(al)$ come funzione

oppure $alloggio_posti_totali(al,\ n)$ - come predicato

$\Rightarrow\ \exists\, num\ (Intero(num)\ \wedge\ alloggio_posti_totali(al,\ num)\ \wedge\ \ \leq (max_ospiti(al),\ n\,)\,)\ \ )$

[Letto.Capienza_minore_uguale_MaxCapienza]

Per ogni *let: Letto, tl: TipoLetto* tali che *(let, tl): let_tl* deve essere vero che *let.capienza* <= *tl.max_capienza*

$\forall\, let,\ tl\ \ (Letto(let)\ \wedge TipoLetto(tl)$

$\wedge\ let_tl(let,\ tl)$

$\Rightarrow\ \ \leq (capienza(let),\ max_capienza(tl))\ )$

[Letto.Num_SceltaPosto_minore_uguale_Capienza]

Per ogni *let: Letto* e ogni istante *t*

- sia SP insieme di tutti *sp: SceltaPosto* tali che
  - *(let, sp): let_sp*
  - *sp.confermato* = True
  - per ogni *sp* in SP esiste *pren: PrenAccettata* tale che
    - *(pren, sp): pren_sp* e *pren.data_inizio* <= *t* < *pren.data_fine*

deve essere vero che

- |SP| <= *let.capienza*

$\forall\, let,\ t\ \ \ (\,(Letto(let)\ \wedge\ DataOra(t)$

$\exists\, SP\ =\ \{\, sp \mid SceltaPosto(sp)$

$\wedge\ \ let_sp(let,\ sp)$

$\wedge\ \ confermato(sp,\ True)$

$\wedge\ \ \exists\, pren\ \ (\,PrenAcettata(pren)$

$\wedge\ pren_sp(pren,\ sp)$

$\wedge\ \ \leq (data_inizio(pren),\ t)$

$\wedge\ \ <\ (t,\ data_fine(pren))\ \ )\ \}$

$\Rightarrow\ \ \leq (|SP|,\ capienza(let))\ \ )$

[Letto.PrenAccettate_non_sovrapposte_se_letto_non_condivisibile]

Per ogni let: Letto tale che (let, tl): let_tip e tl.condivisibile = False

non devono esistere

- pren1: PrenAccettata e pren2: PrenAccettata tali che
  - (pren1, let): pren_let

- (pren2, let): pren_let
- sovrapposti_DataOra(pren1.data_inizio, pren1.data_fine, pren2.data_inizio, pren2.data_fine) = True

$$\forall\, let,\ tl \quad (\ (Letto(let) \wedge TipoLetto(tl)$$
$$\wedge\ let_tip(let,\ tl)$$
$$\wedge\ condivisibile(tl,\ False))$$
$$\Rightarrow\ \neg\exists\, pren_1,\ pren_2\ \ (PrenAccettata(pren_1) \wedge PrenAccettata(pren_2)$$
$$\wedge\ pren_let(pren_1,\ let) \wedge pren_let(pren_2,\ let)$$
$$\wedge\ \ pren_1 \neq pren_2$$
$$\wedge\ \ sovrapposti_DataOra($$
$$data_inizio(pren_1),\ data_fine(pren_1),$$
$$data_inizio(pren_2),\ data_fine(pren_2),\ \ True)\ \ )\ \ )$$

[Prenotazione.DataInizio_minore_DataFine]

Per ogni *pren: Prenotazione* deve essere vero che *pren.data_inizio* < *pren.data_fine*

$$\forall\, pren\ (\ Prenotazione(pren)\ \Rightarrow\ \ <\ (data_inizio(pren),\ data_fine(pren))$$

[Prenotazione.Letti_assegnate_appartengono_allo_stesso_Alloggio]

Per ogni *pren: Prenotazione, let1: Letto, let2: Letto* tali che

- *(pren, let1): pren_let* e *(pren, let2): pren_let*

deve essere vero che

- esiste *al: Alloggio, cam: Camera* tali che
  - *(al, cam): al_cam* ,
  - *(cam, let1): cam_let*
  - *(cam, let2): cam_let*

  oppure
- esiste *al: Alloggio, cam1: Camera, cam2: Camera* tali che
  - (al, cam1): al_cam
  - (al, cam2): al_cam
  - cam1 =/= cam2
  - (cam1, let1): cam_let
  - (cam2, let2): cam_let

$$\forall\, pren,\ let_1,\ let_2\ \ (\ Prenotazione(pren)$$
$$\wedge\ Letto(let_1)$$
$$\wedge\ Letto(let_2)$$
$$\wedge\ pren_let(pren,\ let_1)$$
$$\wedge\ \ pren_let(pren,\ let_2)$$
$$\Rightarrow [\ \ (\ \exists\, al,\ cam\ \ (\ Alloggio(al)$$
$$\wedge\ Camera(cam)$$

$\wedge\ al_cam(al,\ cam)$

$\wedge\ cam_let(cam,\ let_1)$

$\wedge\ cam_let(cam,\ let_2)\ )$

$\vee$

$(\ \exists\ al,\ cam_1,\ cam_2\ (\ Alloggio(al)$

$\wedge\ Camera(cam_1)$

$\wedge\ Camera(cam_2)$

$\wedge\ al_cam(al,\ cam_1)$

$\wedge\ al_cam(al,\ cam_2)$

$\wedge\ cam_1 \neq cam_2$

$\wedge\ cam_let(cam_1,\ let_1)$

$\wedge\ cam_let(cam_2,\ let_2)\ )\ ]\ )$

Tipo **PeriodoChiusura**:
data_inizio: Data
data_fine: Data
giorno_settimana: GiornoSettimana [0..7]

[PeriodoChiusura.GiornoSettimana_diversi]

Per ogni p: PeriodoChiusura tale che |p.giorno_settimana| > 1 deve essere vero che

- per ogni g1: GiornoSettimana e g2: GiornoSettimana tali che
  - g1 in p.giorno_settimana e g2 in p.giorno_settimana deve essere vero che
    - g1 ≠ g2

Non so come scriverlo in FOL.
Se scrivo cosi puo succedere che g1 è lunedi e g2 è lunedi e quindi non puo essere nel conseguente g1 ≠ g2

$\forall\ p,\ g_1,\ g_2\ (\ PeriodoChiusura(p)\ \wedge\ GiornoSettimana\ (g_1)\ \wedge\ GiornoSettimana(g_2)$

$\wedge\ giorno_settimana(p,\ g_1)\ \wedge\ giorno_settimana(p,\ g_1)$

$\Rightarrow\ g1\ \neq\ g2\ )$

[PeriodoChiusura.Inizio_minore_Fine]

Per ogni p: PeriodoChiusura deve essere vero che p.data_inizio < p.data_fine

$\forall\ p\ (\ PeriodoChiusura(p)\ \Rightarrow\ <\ (data_inizio(p),\ data_fine(p))\ )$

[PeriodoChiusura.Opzionale_GiornoSettimana]

Per ogni p: PeriodoChiusura

- se p.giorno_settimana non è valorizzato
  - per ogni d: Data tale che
    - p.data_inizio <= d < p.data_fine
      - data_chiusa_da_periodoChiusura(p, d) = True
- se |p.giorno_settimana| > 0
  - per ogni d: Data tale che
    - p.data_inizio <= d < p.data_fine e d.giorno_settimana ∈ p.giorno_settimana

            - data_chiusa_da_periodoChiusura(p, d) = True
    - per ogni d: Data tale che
        - p.data_inizio <= d < p.data_fine e d.giorno_settimana ∉ p.giorno_settimana
            - data_chiusa_da_periodoChiusura(p, d) = False

$$\forall p\ (PeriodoChiusura(p) \wedge \neg\exists g\ (GiornoSettimana(g) \wedge giorno_settimana(p, g))$$

$$\Rightarrow \forall d\ \ (Data(d)$$

$$\wedge\ \ \leq (data_inizio(p), d)\ \wedge\ \ < (d, data_fine(p))$$

$$\Rightarrow data_chiusa_da_periodoChiusura(p, d, True)\ )\ )$$

$$\forall p\ (PeriodoChiusura(p) \wedge \exists g\ (GiornoSettimana(g) \wedge giorno_settimana(p, g))$$

$$\Rightarrow \forall d\ \ (Data(d)$$

$$\wedge\ \ \leq (data_inizio(p), d)\ \wedge\ \ < (d, data_fine(p))$$

$$\wedge\ \exists g\ \ (GiornoSettimana(g)$$

$$\wedge\ giorno_di_data(d, g)$$

$$\wedge\ giorno_settimana(p, g)\ )$$

$$\Rightarrow data_chiusa_da_periodoChiusura(p, d, True)\ )\ )$$

$$\forall p\ (PeriodoChiusura(p) \wedge \exists g\ (GiornoSettimana(g) \wedge giorno_settimana(p, g))$$

$$\Rightarrow \forall d\ (Data(d)$$

$$\wedge\ \ \leq (data_inizio(p), d)\ \wedge\ \ < (d, data_fine(p))$$

$$\wedge\ \neg\exists g\ \ (GiornoSettimana(g)$$

$$\wedge\ giorno_di_data(d, g)$$

$$\wedge\ giorno_settimana(p, g)\ )$$

$$\Rightarrow data_chiusa_da_periodoChiusura(p, d, False)\ )\ )$$

→giorno_di_data(.)

[PeriodoChiusura.Periodi_non_sovrapposti]

Per ogni al:Alloggio, p1: PeriodoChiusura, p2: PeriodoChiusura
tali che p1 è in al.periodo_chiusura e p2 è in al.periodo_chiusura e p1 ≠ p2

- deve essere vero che
    - *non_sovrapposti_Data*(
        p1.data_inizio,
        p1.data_fine,
        p2.data_inizio,
        p2.data_fine) = True

$$\forall al, p_1, p_2\ (Alloggio(al) \wedge PeriodoChiusura(p_1) \wedge PeriodoChiusura(p_2)$$

$$\wedge\ periodo_chiusura(al, p_1)$$

$$\wedge\ periodo_chiusura(al, p_2)$$

$$\wedge\ p_1 \neq p_2$$

$\Rightarrow\ non_sovraposti_Data($

$data_inizio(p_1),\ data_fine(p_1),$

$data_inizio(p_2),\ data_fine(p_2),\ \ True)\ \ )\ \ )$

Tipo **ChiusuraStraordinaria**:
data_inizio: Data
data_fine: Data
motivo: Stringa[0..1]

[ChiusuraStraordinaria.Inizio_minore_Fine]

Per ogni cs: ChiusuraStraordinaria deve essere vero che cs.data_inizio < cs.data_fine

$\forall\ cs\ (Chiusurastraordinaria(cs)\ \Rightarrow\ \ <\ (data_inizio(cs),\ data_fine(cs))\ )$

[ChiusuraStraordinaria.chiusure_non_sovrapposte]

Per ogni al:Alloggio, cs1: ChiusuraStraordinaria, cs2: ChiusuraStraordinaria tali che cs1 è in al.chiusura_straordinaria e cs2 è in al.chiusura_straordinaria e cs1 ≠ cs2

- deve essere vero che
  - *non_sovrapposti_Data*(
    cs1.data_inizio,
    cs1.data_fine,
    cs2.data_inizio,
    cs2.data_fine) = True

$\forall\ al,\ cs_1,\ cs_2\ (\ Alloggio(al)\ \wedge\ ChiusuraStraordinaria(cs_1)\ \wedge\ ChiusuraStraordinaria(cs_2)$

$\wedge\ \ chiusura_straordinaria(al,\ cs_1)$

$\wedge\ chiusura_straordinaria(al,\ cs_2)$

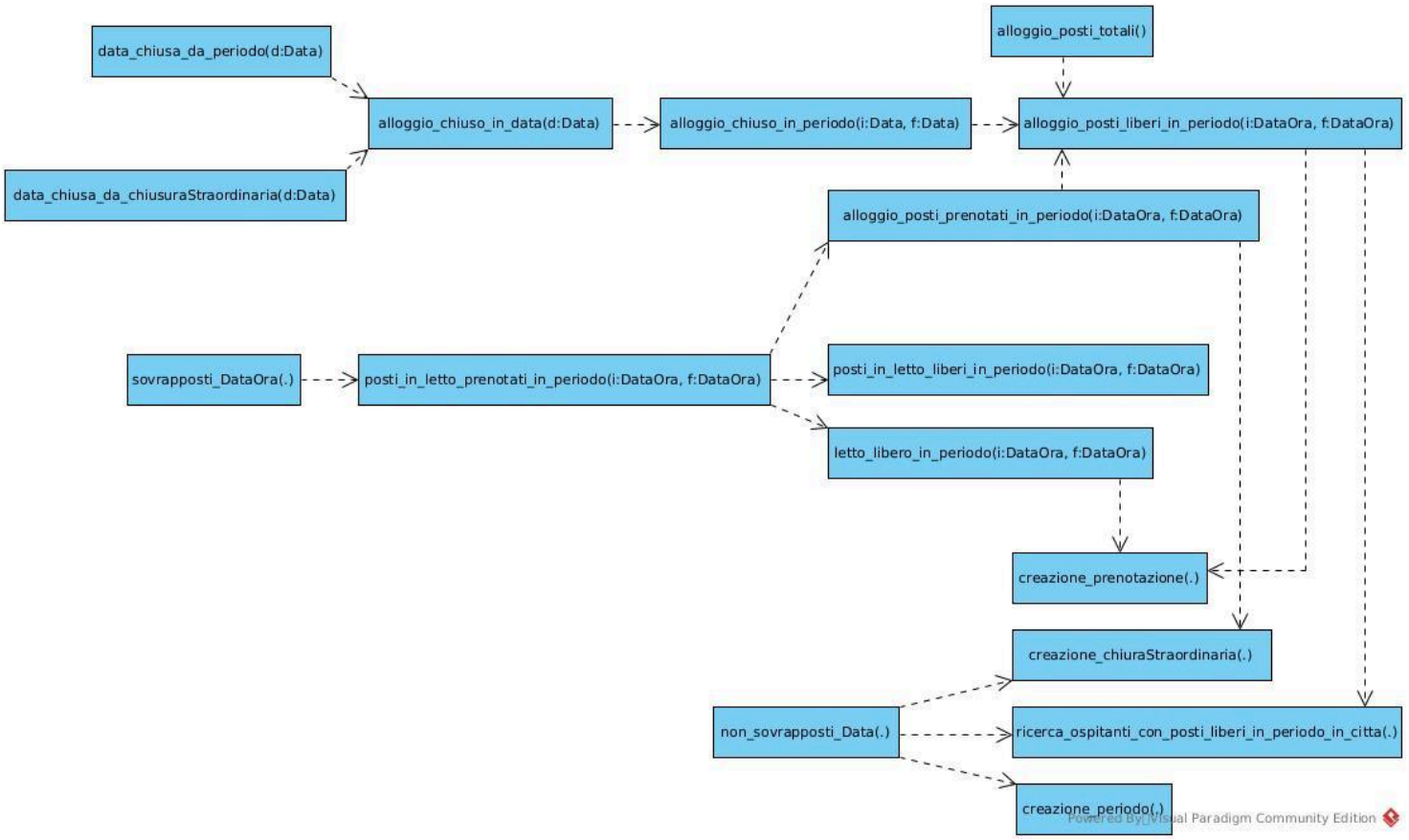
$\wedge\ cs_1 \neq cs_2$

$\Rightarrow\ non_sovraposti_Data($

$data_inizio(cs_1),\ data_fine(cs_1),$

$data_inizio(cs_2),\ data_fine(cs_2),\ \ True)\ )$

Schema che mostra come sono collegati le operazioni ausiliarie, le operazioni di classe e alcuni use-case.
So che nelle specifiche ufficiali uno schema del genere non va incluso ma mi è stato molto utile per costruire la catena di ragionamento.



Operazioni di classe Utente

eta(): Intero > 0

precondizioni:

- *this.data_nascita* < adesso

postcondizioni:

- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:
  - num_anni = *adesso.anno* - *this.data_nascita.anno*

  result = num_anni

Pre:

$$\exists\ d_nasc,\ anno_nasc,\ anno_ades$$

$$(\ Data(d_nasc)$$

$$\wedge\ data_nascita(this,\ d_nasc)$$

$$\wedge\ Intero(anno_nasc)\ \wedge\ anno(d_nasc,\ anno_nasc)$$

$$\wedge\ Data(adesso)$$

$$\wedge\ Intero(anno_ades)\ \wedge\ anno(adesso,\ anno_ades)$$

$$\wedge\ anno_nasc\ <\ anno_ades\ )$$

Post:

- *L'operazione non modifica il livello estensionale*
- *Il risultato result è così definito*

$$result\ =\ anno_ades\ -\ anno_nasc$$

Operazioni di classe Alloggio

Operazioni ausiliarie

alloggio_chiuso_in_data(d: Data): Booleano

precondizioni: nessuna
postcondizioni:

- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:
  - Sia P insieme di tutti p: PeriodoChiusura tali che p è in this.periodo_chiusura
  - Sia CS insieme di tutti cs: ChiusuraStraordinaria tali che cs in in this.chiusura_straordinaria
  - result = True se e solo se
    - esiste p in P tale che
      - data_chiusa_da_periodoChiusura(p, d) = True

      OR
    - esiste cs in CS tale che
      - data_chiusa_da_chiusuraStraordinaria(cs, d) = True

Pre: nessuna
Post:

- *L'operazione non modifica il livello estensionale*
- *Il risultato result è così definito*

$$
\begin{aligned}
& result = True \Leftrightarrow (\\
& \quad (\exists\, p\ (PeriodoChiusura(p) \\
& \qquad \wedge\ periodo_chiusura(this,\ p) \\
& \qquad \wedge\ data_chiusa_da_periodoChiusura(p,\ d,\ True)\ ) \\
& \quad \vee \\
& \quad (\exists\, cs\ (ChiusuraStraordinaria(cs) \\
& \qquad \wedge\ periodo_chiusura(this,\ cs) \\
& \qquad \wedge\ data_chiusa_da_chiusuraStraordinaria(cs,\ d,\ True)\ )\ )
\end{aligned}
$$

 # oppure con insiemi

$$
\begin{aligned}
& sia\ P = \{\, p \mid PeriodoChiusura(p) \wedge periodo_chiusura(this, p) \} \\
& sia\ CS = \{ cs \mid ChiusuraStraordinaria(cs) \wedge chiusura_straordinaria(this,\ cs) \} \\
& result = True \Leftrightarrow \\
& \quad [\, (\exists\, p\ (p \in P \wedge data_chiusa_da_periodoChiusura(p,\ d,\ True)\ ) \\
& \qquad \vee \\
& \quad (\exists\, cs\ (cs \in CS \wedge data_chiusa_da_chiusuraStraordinaria(cs,\ d,\ True))\ ]
\end{aligned}
$$

alloggio_chiuso_in_periodo(d_in: Data, d_fin: Data): Booleano

precondizioni:

- d_in < d_fin

postcondizioni:

- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:
  - result = True se e solo se

- esiste d: Data tale che
  - d_in <= d < d_fin e this.alloggio_chiuso_in_data(d) = True

Pre:

$< (d_in,\ d_fin)$

Post:

- *L'operazione non modifica il livello estensionale*
- *Il risultato result è così definito*

$$result = True \Leftrightarrow$$
$$\exists\, d\ \ (\,Data(d)$$
$$\wedge\ \ \leq (\,d_in,\ d) \wedge\ \ < (d,\ d_fin)$$
$$\wedge\ alloggio_chiuso_in_data(this,\ d,\ True)\,)$$

→ alloggio_posti_liberi_in_periodo(.)

alloggio_posti_totali(): Intero > 0

precondizioni:

- esiste almeno un link (this,cam): al_cam, cam: Camera tale che esiste (cam, let): cam_let, let: Letto

postcondizioni:

- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:
  - Sia C insieme di tutti cam: Camera tali che (this, cam): al_cam
  - Per ogni cam in C
    - sia L insieme di let: Letto tali che che
      - (cam, let): cam_let, cam in C
    - sia *sum_posti* la somma di tutti let.capienza per ogni let in L
  - Sia sum_tot la somma di di tutti *sum_posti per ogni* cam in C

  result = sum_tot

Pre:

$$\exists\, cam,\ let\quad (\,Camera(cam)\ \wedge\ Letto(let)$$
$$\wedge\ cam_let(cam,\ let)\,)$$
$$\wedge\ al_cam(this,\ cam)$$

Post:

- *L'operazione non modifica il livello estensionale*
- *Il risultato result è così definito*

 # non so come scrivere in FOL puro le sommatorie

$$C = \{cam \mid Camera(cam)\ \wedge al_cam(this,\ cam)\}$$
$$\forall\ cam\,(\,cam \in C \Rightarrow$$
$$(\,L = \{\,let \mid Letto(let)\ \wedge\ cam_let(cam,\ let)\,\}$$
$$\wedge$$
$$sum_posti(cam) = \sum_{let \in L} capienza(let)\,)\ )$$

$$Sia\ \ sum_tot = \sum_{cam \in C} sum_posti(cam)$$

$$result = sum_tot$$

→Diagramma UML

alloggio_posti_prenotati_in_periodo(d_in: DataOra, d_fin: DataOra): Intero >= 0

precondizioni:
- d_in < d_fin

postcondizioni:
- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:
  - Sia C insieme di tutti cam: Camera tali che
    - (this, cam): al_cam
  - Per ogni cam in C
    - sia L insieme di let: Letto tali che che
      - (cam, let): cam_let, cam in C
    - sia sum_posti = somma di
      - let.posti_in_letto_prenotati_in_periodo(d_in, d_fin) per ogni let in L
  - sia *sum_tot* = somma di tutti *sum_posti* per ogni cam in C

  result = sum_tot

Pre:

$< (d_in,\ d_fin)$

Post:
- *L'operazione non modifica il livello estensionale*
- *Il risultato result è così definito*

$$Sia\ C\ =\ \{cam \mid Camera(cam)\ \wedge\ al_cam(this,\ cam)\}$$

$$\forall\ cam\ \ (cam \in C\ \Rightarrow$$

$$(\ L\ =\ \{\ let \mid Letto(let)\ \wedge\ cam_let(cam,\ let)\ \}$$

$$\wedge$$

$$sum_posti(cam)\ =\ \sum_{let \in L} posti_in_letto_prenotati_in_periodo(let,\ d_in,\ d_fin)\ )$$

$$sia\ \ sum_tot\ =\ \sum_{cam \in C} sum_posti(cam)$$

$$result\ =\ sum_tot$$

→creazione_chiusuraStraordinaria(.)

Operazioni di classe

alloggio_posti_liberi_in_periodo(d_in: DataOra, d_fin: DataOra): Intero >= 0

precondizioni:
- d_in < d_fin

postcondizioni:
- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:
  - se this.alloggio_chiuso_in_periodo(d_in.**data**, d_fin.**data**) = True
    - result = 0
  - altrimenti
    - result = this.alloggio_posti_totali() - this.alloggio_posti_prenotati_in_periodo(d_in, d_fin)

Pre:

$< (d_in,\ d_fin)$

Post:

- L'operazione non modifica il livello estensionale
- Il risultato result è così definito

$$result = 0$$

$$\Leftrightarrow alloggio_chiuso_in_periodo(this, data_di_DataOra(d_in), data_di_DataOra(d_fin), True)$$

$$result = alloggio_posti_totali(this) - alloggio_posti_prenotati_in_periodo(this, d_in, d_fin)$$

$$\Leftrightarrow alloggio_chiuso_in_periodo(this, data_di_DataOra(d_in), data_di_DataOra(d_fin), False)$$

→ data_di_DataOra(.)

→ creazione_prenotazione(.)

→ ricerca_ospitanti_con_posti_liberi_in_periodo_in_citta(.)

Operazioni di classe Letto

Operazioni ausiliarie

posti_in_letto_prenotati_in_periodo(d_in: DataOra, d_fin: DataOra): Intero >= 0

precondizioni:

- d_in < d_fin

postcondizioni:

- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:
  - sia tl: TipoLetto tale che
    - (this, tl): let_tl
  - sia P insieme di pren: PrenAccettata tali che
    - (pren, this): pren_let
    - sovrapposti_DataOra(pren.data_inizio, pren.data_fine, d_in, d_fin) = True
  - se tl.condivisibile = False
    - se |P| > 0
      - result = this.capienza
    - altrimenti
      - result = 0
  - se tl.condivisibile = True
    - sia SP insieme di *sp: SceltaPosto* tali che
      - (sp, this): let_sp
      - sp.confermato = True
      - e per ogni sp in SP esiste pren in P tale che
        - (sp, pren): pren_sp
    - result = |SP|

Pre:

$$< (d_in, d_fin)$$

Post:

- *L'operazione non modifica il livello estensionale*
- *Il risultato result è così definito*

$$\exists\, tl\ (TipoLetto(tl) \wedge let_tl(this, tl))$$

$$sia\ P = \{ pren \mid PrenAccettata(pren)$$

$$\wedge\ pren_let(pren,\ this)$$

$$\wedge\ \exists\, iniz, fin\ (\ DataOra(iniz)$$

$$\wedge\ DataOra(fin)$$

$$\wedge\ data_inizio(pren,\ iniz)$$

$$\wedge\ data_fine(pren,\ fin)$$

$$\wedge\ sovrapposti_DataOra(\ iniz,\ fin,\ \ d_in,\ d_fin,\ \ True)\ \}$$

$$condivisibile(tl,\ False)\ \Rightarrow [(\ P \neq\ \emptyset\ \Rightarrow result\ =\ capienza(this))$$

$$\vee$$

$$P\ =\ \emptyset\ \Rightarrow result\ =\ 0)\,]$$

$$sia\ SP\ =\ \{\ sp\ |\ SceltaPosto(sp)$$

$$\wedge\ let_sp(this,\ sp)$$

$$\wedge\ confermato(sp,\ True)$$

$$\wedge\ \exists\, pren\ \ (pren \in P\ \wedge\ \ pren_sp(pren,\ sp))\ \}$$

$$condivisibile(tl,\ True)\ \ \Rightarrow\ result\ =\ |SP|$$

→ alloggio_posti_prenotati_in_periodo(.)

→Diagramma UML

Operazioni di classe

posti_in_letto_liberi_in_periodo(d_in: DataOra, d_fin: DataOra): Intero >= 0

precondizioni:

- d_in < d_fin

postcondizioni:

- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:
  - result = this.capienza - this.posti_in_letto_prenotati_in_periodo(d_in, d_fin)

Pre:

$$<\ (d_in,\ d_fin)$$

Post:

- *L'operazione non modifica il livello estensionale*
- *Il risultato result è così definito*

$$result\ =\ capienza(this)\ -\ posti_in_letto_prenotati_in_periodo(this,\ d_in,\ d_fin\,)$$ # come funzione

letto_libero_in_periodo(d_in: DataOra, d_fin: DataOra): Booleano

precondizioni:

- d_in < d_fin

postcondizioni:

- operazione non fa side-effect
- viene restituito il valore "*result*" definito come segue:

- Se e solo se this.posti_in_letto_prenotati_in_periodo(d_in, d_fin) = 0
  - result = True
- altrimenti
  - result = False

Pre:

$< (d_in,\ d_fin)$

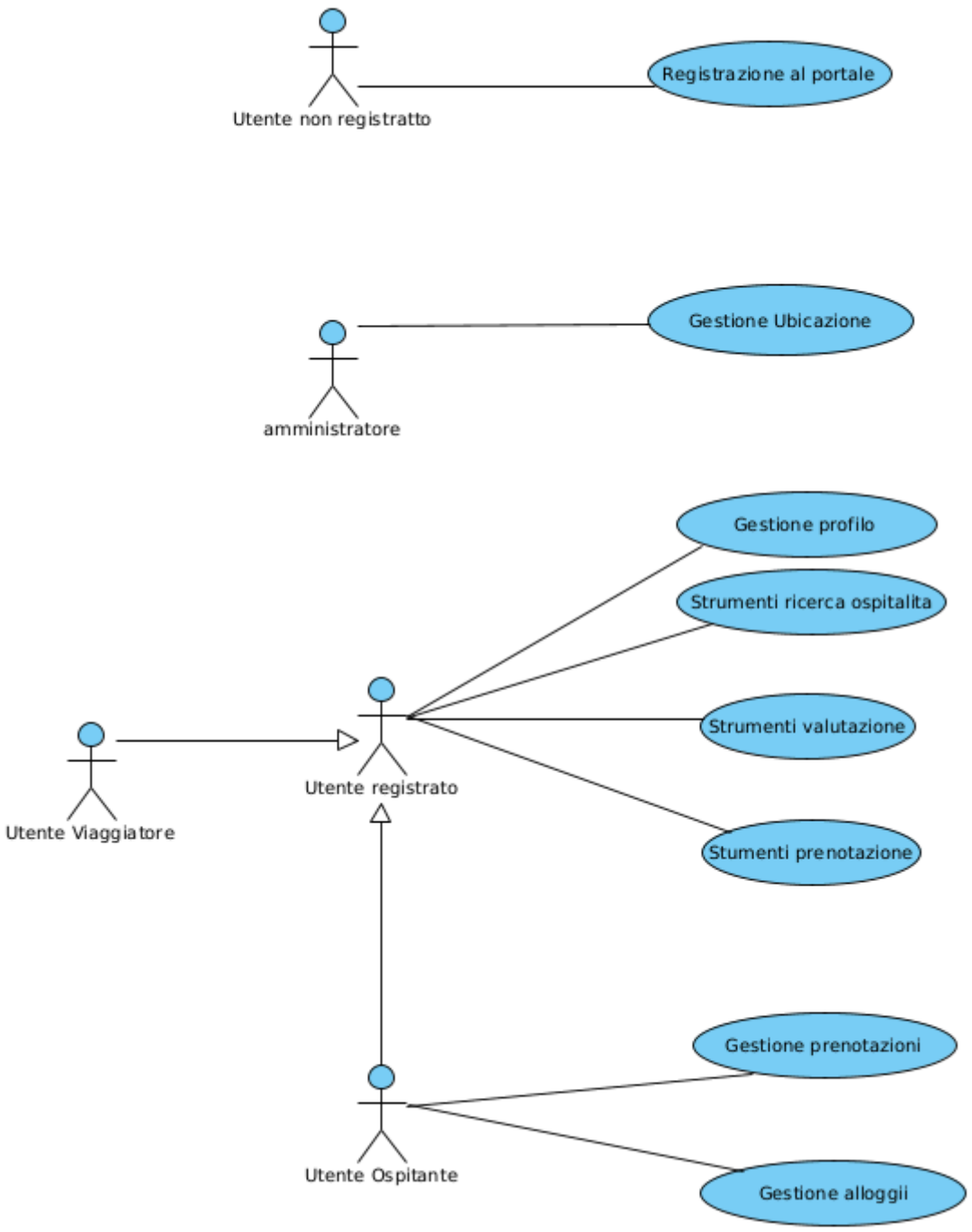
Post:

- *L'operazione non modifica il livello estensionale*
- *Il risultato result è così definito*

$$result\ =\ True\ \Leftrightarrow posti_in_letto_prenotati_in_periodo(this,\ d_in,\ d_fin,\ 0\ )$$ # come predicato

→creazione_prenotazione(.)

1.2. Diagramma Use-Case



→Diagramma UML

1.2.1. Operazioni di use-case Registrazione al portale

registrazione_utente(

nom: Stringa,
cogn: Stringa,
sesso: Sesso,
dat_nasc: Data,
offre_ospit: Booleano,
cit: Citta
) : Utente

precondizioni:

- dat_nasc < adesso

postcondizioni:

- viene creato e restituito un nuovo oggetto *result: Utente* con valori
  - *result.nome* = nom
  - *result.cognome* = cogn
  - *result.sesso* = sesso
  - *result.data_nascita* = dat_nasc
  - *result.offre_ospitalita* = offre_ospit
  - viene creato link *(result, cit): residenza*

Pre:

$$\exists\, anno_nasc,\ anno_ades\ \ (\ Intero(anno_nasc)$$
$$\wedge\ \ anno(dat_nasc,\ \ anno_nasc\ )$$
$$\wedge\ \ Intero(anno_ades)$$
$$\wedge\ \ anno(adesso,\ anno_ades)$$
$$\wedge\ \ anno_nasc\ \ <\ \ anno_ades\ )$$

Post:

$$\exists\, ut\ \ (\ Utente(ut)$$
$$\wedge\ nome(ut,\ nom)$$
$$\wedge\ cognome(ut,\ cogn)$$
$$\wedge\ sesso(ut,\ sesso)$$
$$\wedge\ data_nascita(ut,\ dat_nasc)$$
$$\wedge\ offre_ospitalita(ut,\ offre_ospit)$$
$$\wedge\ residenza(ut,\ cit)\ \ )$$
$$result\ =\ ut$$

→Diagramma UML

1.2.2. Operazioni di use-case Gestione alloggi

registrazione_alloggio(

osp: Utente,
num_adult: Intero >= 0,
num_bamb: Intero >= 0,
max_osp: Intero > 0,
P: PeriodoChiusura [0..*],
CS: ChiusuraStraordinaria [0..*],
cit: Citta,
dist_da_cent: Intero > 0,
tip_al: TipoAlloggio,
tip_staz: TipoStazione,
dist_da_staz: Intero > 0
) : Alloggio

precondizioni: nessuna
postcondizione:

- viene creato e restituito un nuovo oggetto *result: Alloggio* con valori
  - *result.num_adulti* = num_adult
  - *result.num_bambini* = num_bamb
  - *result.max_ospiti* = max_osp
  - se |P| > 0
    - *result.periodo_chiusura* = P

    altrimenti
    - *result.periodo_chiusura* non è valorizzato
  - se |CS| > 0
    - *result.chiusura_straordinaria* = CS

    altrimenti
    - *result.chiusura_straordinaria* non è valorizzato
  - vengono creati i link:
    - *(result, osp): ospitalita*
    - *(result, tip_al): al_tip*
    - *(result, tip_staz): al_st* con valore *(result, tip_staz).dist_da_staz_metri* = dist_da_staz
    - *(result, cit): cit_al* con valore *(result, cit).dist_dal_centro_metri* = dist_da_cent

$Pre:\ nessuna$

$Post:$

$$
\begin{aligned}
&\exists\, al\ \ (\, Alloggio(al) \\
&\qquad \wedge\ num_adulti(al,\ num_adult) \\
&\qquad \wedge\ num_bambini(al,\ num_bamb) \\
&\qquad \wedge\ max_ospiti(al,\ max_osp) \\
&\qquad \wedge\ \forall\, p\ (\, p \in P \Rightarrow periodo_chiusura(al,\ p)\,) \\
&\qquad \wedge\ \forall\, cs\ (\, cs \in SC \Rightarrow\ chiusura_straordinaria(al,\ cs)\,) \\
&\qquad \wedge\ ospitalita(al,\ osp) \\
&\qquad \wedge\ al_tip(al,\ tip_al) \\
&\qquad \wedge\ al_st(al,\ tip_staz) \\
&\qquad \wedge\ dist_da_staz_metri(al,\ tip_staz,\ dist_da_staz) \\
&\qquad \wedge\ cit_al(al,\ cit) \\
&\qquad \wedge\ dist_dal_centro_metri(al,\ cit,\ dist_da_cent)
\end{aligned}
$$

$result = al$

creazione_camera_in alloggio(al: Alloggio, nom: Stringa) : Camera

precondizione:

- non esiste un altro oggetto *cam : Camera* con valore *cam.nome* = *nom* tale che *(al, cam): al_cam*

postcondizioni:

- viene creato e restituito un nuovo oggetto *result: Camera* con valori
  - *result.nome* = nom
  - viene creato il link *(al, cam): al_cam*

`Pre:`

$\neg\exists\, cam\ (\, Camera(cam)\ \wedge\ nome(cam,\, nom)\ \wedge\ al_cam(cam,\, al)\,)$

`Post:`

$\exists\, cam\ (\, Camera(cam)\ \wedge\ nome(cam,\, nom)\ \wedge\ al_cam(cam,\, al)\,)$

$return = cam$

→Diagramma UML

creazione_unita_letto(

al: Alloggio,
cam: Camera,
cap: Intero > 0,
tip_let: TipoLetto
) : Letto

precondizione: nessuna

postcondizioni:

- viene creato e restituito un nuovo oggetto *result: Letto* con valori
  - *result.capienza* = cap
  - vengono creati i link:
    - *(result, cam): cam_let*
    - *(result, tip_let): let_tl*

Pre: nessuna

Post:

$\exists\, let\ (\, Letto(let)\ \wedge\ nome(let,\, nom)\ \wedge\ al_cam(let,\, al)\,)$

$result = let$

Ospitante ha la possibilita di dichiarare delle date o dei periodi nei quali non è disponibile a ricevere ospiti,

Tipo **PeriodoChiusura**:
data_inizio: Data
data_fine: Data
giorno_settimana: GiornoSettimana [0..7]

creazione_periodo_chiusura(

al: Alloggio,
d_in: Data,
d_fin: Data,
G: GiornoSettimana [0..7]
)

precondizioni:

- d_in < d_fin

- se |G| > 1 deve essere vero che
    - per ogni g1: GiornoSettimana e g2: GiornoSettimana tali che g1 e g2 in G deve essere vero che
        - g1 ≠ g2
- per ogni *pc* in *al.periodo_chiusura* deve essere vero che
    - *non_sovrapposti_Data*(
      pc.data_inizio,
      pc.data_fine,
      d_in,
      d_fin) = True

postcondizioni:

- viene creato un nuovo valore *pc': PeriodoChiusura* con campi
    - *pc'.data_inizio* = d_in
    - *pc'.data_fine* = d_fin
    - *pc'.giorno* = G
- viene aggiunto *pc'* ai valori *al.periodo_chiusura*

Pre:

$$< (d_in,\ d_fin)$$

$$\wedge\ \forall g_1, g_2\ [(g_1 \in G \wedge g_2 \in G) \Rightarrow g_1 \neq g_2]$$

$$\wedge\ \forall pc,\ PeriodoChiusura(pc)\ \wedge\ periodo_chiusura(al,\ pc)$$

$$\Rightarrow non_sovrapposti_Data(\ data_inizio(pc),\ data_fine(pc),\ d_in,\ d_fin,\ True)$$

Post:

$$sia\ PC_{pre} = periodo_chiusura(al)_{OLD}$$

$$\exists\ pc',\ PeriodoChiusura(pc')$$

$$\wedge\ data_inizio(pc',\ d_in)$$

$$\wedge\ data_fine(pc',\ d_fin)$$

$$\wedge\ \forall g\ (g \in G \Rightarrow giorno_settimana(pc',\ g))$$

$$periodo_chiusura(al) = PC_{pre} \cup pc'$$

eliminazione_periodo_chiusura(
al: Alloggio,
pc: PeriodoChiusura
)

precondizioni:

- *pc* è uno dei valori *in al.periodo_chiusura*

postcondizioni:

- viene rimosso *pc* dal *al.periodo_chiusura*

Pre:

$$\exists\ pc\ (PeriodoChiusura(pc) \wedge periodo_chiusura(al,\ pc))$$

Post:

$$sia\ PC_{pre} = periodo_chiusura(al)_{OLD}$$

```
# versione 1
```

$$periodo_chiusura(al) = PC_{pre} \setminus pc$$

versione 2

$\forall pc' \; [\, periodo_chiusura(al, pc') \;\Leftrightarrow\; (pc' \neq pc \;\wedge\; pc' \in PC_{pre}) \,]$

creazione_chiusuraStraordinaria(

al: Alloggio,
d_in: Data,
d_fin: Data,
motiv: Stringa[0..1]
)

precondizioni:

- d_in < d_fin
- al.alloggio_posti_prenotati_in_periodo(d_in, d_fin) = 0
- per ogni *cs* in *al.chiusura_straordinaria* deve essere vero che
  - *non_sovrapposti_Data*(
    cs.data_inizio,
    cs.data_fine,
    d_in,
    d_fin) = True

postcondizioni:

- viene creato un nuovo valore *cs: ChiusuraStraordinaria* con campi
  - *cs.data_inizio* = d_in
  - *cs.data_fine* = d_fin
  - *se motivo è valorizzato*
    - *cs.motivo = motiv*

    altrimenti
    - *cs.motivo non è valorizzato*
- viene aggiunto *cs* ai valori *al.chiusura_straordinaria*

Pre:

$< (d_in,\; d_fin)$

$\wedge\; alloggio_posti_prenotati_in_periodo(al,\; d_in,\; d_fin,\; 0)$

$\wedge\; \forall cs \;\; (\, ChiusuraStraordinaria(cs) \;\wedge\; chiusura_straordinaria(al,\; cs)\,)$

$\Rightarrow\; non_sovrapposti_Data(\, data_inizio(cs),\; data_fine(cs),\;\; d_in,\; d_fin,\;\; True)$

Post:

$sia\; CS_{pre} = chiusura_straordinaria(al)_{OLD}$

$\exists\, cs \;\; (\, ChiusuraStraordinaria(cs)$

$\wedge\;\; data_inizio(cs,\; d_in)$

$\wedge\;\; data_fine(cs,\; d_fin)$

$\wedge\;\; \exists\, motiv \;\Rightarrow\; motivo(cs,\; motiv) \;\;\;)$

$chiusura_straordinaria(al) \;\; = CS_{pre} \;\cup\;\; cs$

eliminazione_chiusura_straordinaria(

al: Alloggio,
cs: ChiusuraStraordinaria
)

precondizioni:

- *cs* è uno dei valori *in al.chiusura_straordinaria*

postcondizioni:

- viene rimosso *cs* dal *al.chiusura_straordinaria*

Pre:

$$\exists\, cs\ \ (\, ChiusuraStraordinaria(cs)\ \wedge\ \ chiusura_straordinaria(al,\ cs)\,)$$

Post:

$$sia\ CS_{pre}\ =\ \ chiusura_straordinaria(al)_{OLD}$$

$$chiusura_straordinaria(al)\ =\ PC_{pre} \setminus\ cs$$

1.2.3. Operazione di use-case Strumenti ricerca alloggio

I viaggiatori che desiderano organizzare un viaggio usufruendo del servizio devono poter interrogare il sistema per ottenere l'insieme di tutti gli **iscritti** della città desiderata che sono disposti a ricevere ospiti nel periodo richiesto e hanno, per le date richieste, posti sufficienti disponibili.

ricerca_ospitanti_con_posti_liberi_in_periodo_in_citta(

num_pers: Intero > 0,
cit: Citta,
d_in: DataOra,
d_fin: DataOra,
) :Uttente [0..*]

precondizioni:

- d_in < d_fin

postcondizioni:

- operazione non modifica i dati
- viene restituito il valore "*result*" definito come segue:
  - sia A insieme di tutti *al: Alloggio* tali che
    - esiste link *(cit, al): cit_al*
  - sia A1 insieme di di tutti *al* in A tali che
    - esiste *ut: Utente* tale che *(ut, al): ospitalita*
  - sia A2 insieme di tutti *al* in A1 tali che
    - *al.alloggio_posti_liberi_in_periodo(d_in, d_fin) >= num_pers*
  - *sia U insieme di tutti ut: Utente tali che*
    - *(al, ut): ospitalita,*
    - *al in A2*

result = U

Pre:

$$<\ (d_in,\ d_fin)$$

Post:

- *L'operazione non modifica il livello estensionale*
- *Il risultato result è così definito*

$$sia\ A\ \ =\ \{\, al\,|\ \ Alloggio(al)$$

$\wedge\ cit_al(al,\ cit)$

$\wedge\ \exists\, ut\ (\ Utente(ut)\ \wedge\ ospitalita(al,\ ut)\ )$

$\wedge\ alloggio_posti_liberi_in_periodo(al,\ d_in,\ d_fin)\ \geq\ num_pers\ \}$ # come funzione

oppure

$\wedge\ \ \geq (alloggio_posti_liberi_in_periodo(al,\ d_in,\ d_fin),\ num_per\ )$ # come predicato

≥ /2

$sia\ U\ =\ \{ut \mid Utente(ut)\ \wedge\ (\exists\, al \in A\ \ ospitalita(al,\ ut))\}$

$result\ =\ U$

→Diagramma UML

1.2.4. Operazione di use-case Strumenti prenotazioni

Una volta scelta una soluzione di alloggio, i viaggiatori possono effettuare la relativa richiesta di prenotazione, specificando tutte le informazioni necessarie (date, persone, scelta dei posti letto, ecc.). é possibile per un viaggiatore prenotare anche per eventuali accompagnatori:

creazione_prenotazione(

org: Utente,
d_in: DataOra,
d_fin: DataOra,
al: Alloggio,
PF: Preferenze [1..*]
) : Prenotazione

precondizioni:

- d_in < d_fin
- *al.alloggio_posti_liberi_in_periodo(d_in, d_fin) >= |PF|*
- esiste *pref* in PF tale che
  - *pref.utente = org*
- per ogni *pref* in PF deve essere vero che
  - *pref.letto.letto_libero_in_periodo(d_in, d_fin) = True*
  - esiste *cam: Camera* tale che
    - *(cam, pref.letto): cam_let*
    - *(al, cam): al_cam*

postcondizioni:

- viene creato e restituito un nuovo oggetto result: Prenotazione con valori
  - *result.data_inizio* = d_in
  - *result.data_fine* = d_fin
  - viene creato un link *(result, org): organizzatore*
  - per ogni *pref* in PF vengono creati i link
    - *(result, pref.utente): visitatore*
    - *(result, pref.letto): pren_let*
    - viene creato un nuovo oggetto *sp: SceltaPosto* con valore
      - *sp.confermato* = False
      - se *pref.posiz è valorizzato*
        - *sp.posizione = pref.posiz*
      - *altrim enti sp.posizione non è valorizzato*
    - vengono creati i link
      - *(pref.utente, sp): ut_sp*
      - *(pref.letto, sp): let_sp*
      - *(result, sp): pren_sp*

Tipo **Preferenze**:
utente: Utente
letto: Letto
posiz: Posizione [0..1]

```
creazione_prenotazione(
      org: Utente,
      d_in: Data,
      d_fin: Data,
      al: Alloggio,
      PF: Preferenze [1..*]        # PF contiene info su chi quale letto ha scelto
      ) : Prenotazione
```

Pre:

$< (d_in,\ d_fin)$

$\wedge\ \geq (alloggio_posti_liberi_in_periodo(al,\ d_in,\ d_fin),\ |PF|)$ # posso scrivere |PF| ?

$\wedge\ \exists\, pref\ (\ pref \in PF\ \wedge\ utente(pref,\ org)\ )$

$\wedge\ \forall\, pref\ (\ pref \in PF\ \Rightarrow$

$\exists\, cam\ (\ Camera(cam)$

$\wedge\ al_cam(al,\ cam)$

$\wedge\ cam_let(cam,\ letto(pref))$

$\wedge\ letto_libero_in_periodo(letto(pref),\ d_in,\ d_fin,\ True)\ )$

Post:

$\exists\, pren\ (\ Prenotazione(pren)$

$\wedge\ data_inizio(pren,\ d_in)$

$\wedge\ data_fine(pren,\ d_fin)$

$\wedge\ organizzatore(pren,\ org)$

$\wedge\ \forall\, pref\ (pref \in PF$

$\Rightarrow\ \wedge\ visitatore(pren,\ utente(pref))$

$\wedge\ pren_let(pren,\ letto(pref))$

$\wedge\ \exists\, sp\ (\ SceltaPosto(sp)$

$\wedge\ confermato(sp,\ False)$

 # versione 1 con equivalenza

$\wedge\ (\ \forall\, pos\ (Posizione(pos)\ \Rightarrow\ (posiz(pref,\ pos)\ \Leftrightarrow\ posizione(sp,\ pos))\ )$

 # oppure versione 2 con implicazioni (quantificatore + condizione su pref) ⇒ (condizione su sp)

$\wedge\ [\ (\ \forall\, pos\ (Posizione(pos)\ \wedge\ posiz(pref,\ pos))\ )\ \Rightarrow\ posizione(sp,\ pos)$

$\wedge$

$(\ \neg\exists\, pos\ (Posizione(pos)\ \wedge\ posiz(pref,\ pos))\ )\ \Rightarrow\ \neg\exists\, posizione(sp,\ pos)\ ]$

 # oppure versione 3 con AND / OR

$\wedge\ [\ (\ \exists\, pos\ (Posizione(pos)\ \wedge\ posiz(pref,\ pos)\ \wedge\ posizione(sp,\ pos)\ )\ )$

$\vee$

$(\neg\exists\, pos\ (Posizione(pos)\ \wedge\ posiz(pref,\ pos)\ \wedge\ \neg\exists\, posizione(sp,\ pos)\ )\ )\ ]$

$\wedge\ ut_sp(sp,\ utente(pref))$

$\wedge\ let_sp(sp,\ letto(pref))$

$\wedge\ pren_sp(sp,\ pren)\ )$

$result\ =\ pren$

→*alloggio_posti_liberi_in_periodo*(.)

→*letto_libero_in_periodo*(.)

1.2.5. Operazione di use-case Gestione prenotazioni

Il relativo ospitante (membro padrone di casa) può decidere liberamente se accettare la richiesta o rifiutarla (in quest'ultimo caso deve specificare una motivazione).

ospitante_accetta_prenotazione(pren: Prenotazione) : PrenAccettata

precondizioni

- *pren* non è istanza ne di *PrenAccettata* ne di *PrenRifiutata*

postcondizioni:

- modifiche ai dati:
  - *pren* diventa istanza di *PrenAccettata* con valori per attributi:
    - *pren.istante_accettazione* = istante di adesso
  - per ogni *sp: SceltaPosto* tale che *(pren, sp): pren_sp*
    - *sp.confermato* = True

$Pre:$

$\neg\ PrenAccettata(pren)\ \wedge\ \neg PrenRifiutata(pren)$

$Post:$

$PrenAcettata(pren)$

$\wedge\ istante_accettazione(pren,\ adesso)$

$\wedge\ \forall\ sp\ \ (SceltaPosto(sp)\ \wedge\ pren_sp(sp,\ pren))\ \Rightarrow\ confermato(sp,\ True)$

$return\ =\ pren$

→Diagramma UML

ospitante_rifiuta_prenotazione(pren: Prenotazione, motiv: Stringa) : PrenRifiutata

precondizioni

- *pren* non è istanza ne di *PrenAccettata* ne di *PrenRifiutata*

postcondizioni:

- modifiche ai dati:
  - *pren* diventa istanza di *PrenRifiutata* con valori:
    - *pren.istante_rifiuto* = istante di adesso
    - *pren.motivazione* = motiv

$Pre:$

$\neg\ PrenAccettata(pren)\ \wedge\ \neg PrenRifiutata(pren)$

$Post:$

$PrenRifiutata(pren)$

$\wedge\ istante_rifiuto(pren,\ adesso)$

$\wedge\ motivazione(pren,\ motiv)$

$result = pren$

1.2.6. Operazione di use-case Strumenti valutazione

Ogni viaggiatore, dopo aver alloggiato presso un iscritto ospitante, deve necessariamente esprimere una valutazione su quest'ultimo, in termini di un intero da 0 (scarso) a 5 (ottimo). In particolare, sebbene quest'ultima possa essere effettuata anche in un tempo successivo, il sistema deve fare in modo che un viaggiatore non possa prenotare più alcuna nuova sistemazione se non ha valutato tutti gli ospitanti presso cui ha alloggiato.

viaggiatore_valuta_ospitante(

pren: PrenAccettata,
viag: Utente,
voto: Voto
)

precondizione:

- esiste link *(viag, pren): visitatore*
- *(viag, pren).feed_su_ospitante* non è valorizzato

postcondizioni:

- viene aggiunto un valore al attributo
  - *(viag, pren).feed_su_ospitante* = voto

$Pre:$

$visitatore(viag, pren)$

$\wedge \neg\exists\, feed, feed_su_ospitante(viag, pren, feed)$

$Post:$

$feed_su_ospitante(viag, pren, voto)$

→Diagramma UML

Ogni ospitante, all'atto della partenza di un ospite, può esprimere una valutazione su quest'ultimo, anche qui in termini di un intero da 0 a 5.

ospitante_valuta_visitatore(

pren: PrenAccettata,
viag: Utente,
osp: Utente,
voto: Voto
)

precondizione:

- esiste link *(viag, pren): visitatore*
- *(viag, pren).feed_su_visitatore* non è valorizzato
- esiste link *(osp, al): ospitalita, al: Alloggio* tale che
  - esiste *(al, cam): al_cam, cam: Camera* tale che
    - esiste *(cam, let): cam_let, let: Letto* tale che
      - *(pren, let): pren_let*

postcondizioni:

- viene aggiunto un valore al attributo
  - *(viag, pren).feed_su_visitatore* = voto

$Pre:$

$visitatore(viag, pren)$

$\wedge \neg\exists\, feed, feed_su_visitatore(viag, pren, feed)$

$\wedge\ \exists\, al\ \ (\, Alloggio(al)$

$\wedge\ ospitalita(al,\ osp)$

$\wedge\ \exists\, cam\ \ (\, Camera(cam)$

$\wedge\ \ al_cam(al,\ cam)$

$\wedge\ \exists\, let\ \ (\, Letto(let)$

$\wedge\ \ cam_let(cam,\ let)$

$\wedge\ \ pren_let(pren,\ let)\,)\ \ )\ \ )$

$Post:$

$feed_su_visitatore(viag,\ pren,\ voto)$

1.2.7. Operazione di use-case Gestione Ubicazione

creazione_tipo_letto(

nome_arr: Stinga,
form_let: Stringa,
max_cap: Intero > 0,
cond: Booleano
) : TipoLetto

precondizioni:

- non esiste un oggetto *tl: TipoLetto* con i valori *tl.nome_arredo* = *nome_arr* e *tl.formato_letto* = *form_let*

postcondizioni:

- viene creato e restituito un nuovo oggetto *result: TipoLetto* con valori
  - *result.nome_arredo* = nome_arr
  - *result.formato_letto* = form_let
  - *result.max_capienza* = max_cap
  - *result.condivisibile* = cond

$Pre:$

$\neg\exists\, tl\ \ (TipoLetto(tl)\ \ \wedge\ nome_arredo(tl,\ nome_arr)\ \ \wedge\ formato_letto(tl,\ form_let))$

$Post:$

$\exists\, tl\ \ (\ TipoLetto(tl)$

$\wedge\ nome_arredo(tl,\ nome_arr)$

$\wedge\ formato_letto(tl,\ form_let)$

$\wedge\ max_capienza(tl,\ max_cap)$

$\wedge\ \ condivisibile(tl,\ cond)\ )$

$result\ \ =\ \ tl$

creazione_tipo_stazione(nom: Stringa) : TipoStazione

precondizioni:

- non esiste un oggetto *st: TipoStazione* con i valori *st.nome* = *nom*

postcondizioni:

- viene creato e restituito un nuovo oggetto *result: TipoStazione* con valori
  - *result.nome* = nom

$Pre:$

$\neg\exists\, st\ (\ TipoStazione(st)\ \wedge nome(st,\ nom)\ )$

$Post:$

$\exists\, st\ (TipoStazione(st)\ \wedge nome(st,\ nom)\ )$

$result\ =\ st$

creazione_tipo_alloggio(nom: Stringa) : TipoAlloggio

precondizioni:

- non esiste un oggetto *al: TipoAlloggio* con i valori *al.nome* = *nom*

postcondizioni:

- viene creato e restituito un nuovo oggetto *result: TipoAlloggio* con valori
  - *result.nome* = nom

$Pre:$

$\neg\exists\, al\ (\ TipoAlloggio(al)\ \wedge nome(al,\ nom)\ )$

$Post:$

$\exists\, al\ (TipoAlloggio(al)\ \wedge nome(al,\ nom)\ )$

$result\ =\ al$

creazione_citta(nom: Stringa, reg: Regione) : Citta

precondizioni:

- non esiste un oggetto *cit: Citta* tale che *(cit, reg): cit_reg* e *cit.nome* = *nom*

postcondizioni:

- viene creato e restituito un nuovo oggetto *result: Citta* con valori
  - *result.nome* = nom
  - viene creato il link *(result, reg): cit_reg*

$Pre:$

$\neg\exists\, cit\ (\ Citta(cit)\ \wedge nome(cit,\ nom)\ \wedge cit_reg(cit,\ reg)\ )$

$Post:$

$\exists\, cit\ (\ Citta(cit)\ \wedge nome(cit,\ nom)\ \wedge cit_reg(cit,\ reg)\ )$

$result\ =\ cit$

creazione_regione(nom: Stringa, naz: Nazione) : Regione

precondizioni:

- non esiste un oggetto *reg: Regione* tale che *(reg, naz): reg_naz* e *reg.nome* = *nom*

postcondizioni:

- viene creato e restituito un nuovo oggetto *result: Regione* con valori
  - *result.nome* = nom
  - viene creato il link *(result, naz): reg_naz*

$Pre:$

$\neg\exists\, reg\ (\ Regione(reg)\ \wedge nome(reg,\ nom)\ \wedge reg_naz(reg,\ naz)\ )$

$Post:$

$\exists\, reg\ (\ Regione(reg)\ \wedge nome(reg,\ nom)\ \wedge reg_naz(reg,\ naz)\ )$

$result\ =\ reg$

creazione_nazione(nom: Stringa) : Nazione

precondizioni:

- non esiste un oggetto *naz: Nazione* con i valori *naz.nome* = *nom*

postcondizioni:

- viene creato e restituito un nuovo oggetto *result: Nazione* con valori
  - *result.nome* = nom

$Pre:$

$$\neg\exists\, naz\ (\, Nazione(naz)\ \wedge\ nome(naz,\ nom)\,)$$

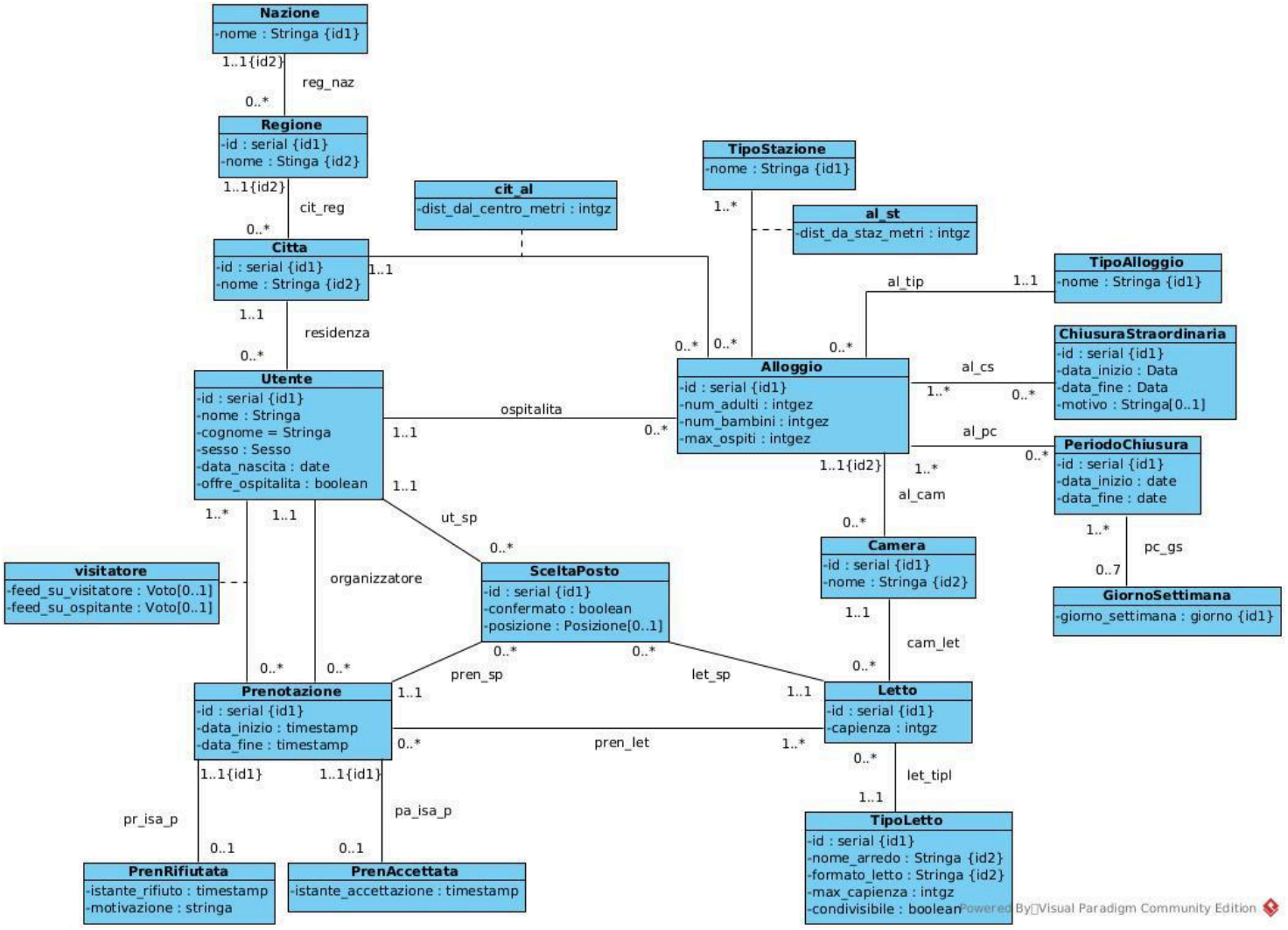
$Post:$

$$\exists\, naz\ (\, Nazione(naz)\ \wedge\ nome(naz,\ nom)\,)$$

$$result\ =\ naz$$

2. Ristrutturazione per Basi Dati

2.1. Diagramma UML ristrutturato per bd



2.2. Specifica dei vincoli esterni

[Prenotazione.Disgiunzione]

Per ogni pren: Prenotazione non devono esistere prenrif: PrenRifiutata e prenac: PrenAccettata tale che

- (pren, prenrif): pf_isa_p e (pren, prenac): pa_isa_p

$$\forall\, pren\ (\ Prenotazione(pren)\ \Rightarrow \neg\, \exists\, prenrif\,,\ prenac\ (\ PrenRifiutata(prenrif)\ \wedge\ pf_isa_p(pren,\ prenrif) \wedge\ PrenAccettata(prenac)\ \wedge\ pa_isa_p(pren,\ prenac)\ )\ )$$

oppure

$$\forall\, prenrif,\ prenac\ (\ (PrenRifiutata(prenrif)\ \wedge\ PrenAccettata(prenac))\ \Rightarrow \neg\exists\, pren\ (Prenotazione(pren)\ \wedge\ pf_isa_p(pren,\ prenrif)\ \wedge\ pa_isa_p(pren,\ prenac)\ )\ )$$

[Prenotazione.organizzatore_isa_visitatore]

Per ogni pren: Prenotazione, ut: Utente tali che (ut, pren): organizzatore deve esiste link (ut, pren): visitatore.

$$\forall\, pren, ut\ (\ (\ Prenotazione(pren)\ \wedge\ Utente(ut)\ \wedge\ organizzatore(ut,\ pren)\ )\ \Rightarrow\ visitatore(ut,\ pren)\ )$$

2.3. Traduzione diretta

2.3.1. Tipi di dato

```
create domain stringa as varchar;

create domain intgz as integer check (value > 0);

create domain intgez as integer check (value >= 0);

create domain voto as integer
   check (value BETWEEN 0 and 5);

create type sesso as
   enum('maschio', 'femmina');

create type giorno as
   enum('Lunedi', 'Martedi', 'Mercoledi', 'Giovedi', 'Venerdi', 'Sabato', 'Domenica');

create type posizione as
   enum('sotto', 'sopra');
```

2.3.2. Schema relazionale

```
create table nazione(
  nome stringa primary key
);

create table regione(
   id serial primary key,
   nome stringa not null,
   nazione stringa not null,
```

```
   unique (nome, nazione),
   foreign key (nazione)
       references nazione (nome)       -- accorpa reg_naz
);

create table citta(
   id serial primary key,
   nome stringa not null,
   regione_id integer not null,
   unique ( nome, regione_id),
   foreign key (regione_id)
       references regione(id)      -- accorpa cit_reg
);

create table utente(
   id serial primary key,
   nome stringa not null,
   cognome stringa not null,
   sesso sesso not null,
   data_nascita date not null,
   offre_ospitalita boolean not null,
   citta_id integer not null,
   foreign key (citta_id)
       references citta(id)        -- accorpa residenza
);

create table tipostazione(
   nome stringa primary key
);

create table tipoalloggio(
   nome stringa primary key
);

create table  alloggio(
   id serial primary key,
   num_bambini intgez not null,
   num_adulti intgez not null,
   max_ospiti intgez not null,
   citta_id integer not null,
   dist_dal_centro_metri intgz not null,
   utente_id integer not null,
```

```sql
   tipoalloggio stringa not null,
   foreign key (citta_id)
       references citta(id),        -- accorpa cit_al
   foreign key (utente_id)
       references utente(id),        -- accorpa ospitalita
   foreign key (tipoalloggio)
       references tipoalloggio(nome)  -- accorpa al_tip
   -- v.inclusione alloggio(id) occorre in al_st(alloggio_id)
);

create table al_st(
   alloggio_id integer not null,
   tipostazione stringa not null,
   primary key( alloggio_id, tipostazione),
   dist_da_staz_metri intgz not null,
   foreign key (alloggio_id)
       references alloggio(id),
   foreign key (tipostazione)
       references tipostazione(nome)
);

create table giornosettimana(
   giorno_settimana giorno primary key
   -- v.inclusione giornosettimana(giorno_settimana) occore in pc_gs(giornosettimana)
);

create table periodochiusura(
   id serial primary key,
   data_inizio date not null,
   data_fine date not null,
   check (data_inizio < data_fine)
   -- v.inclusione periodochiusura(id) occorre in al_pc(periodochiusura_id)
);

create table pc_gs(
   periodochiusura_id integer not null,
   giornosettimana giorno not null,
   primary key (periodochiusura_id, giornosettimana),
   foreign key (periodochiusura_id)
       references periodochiusura(id),
   foreign key (giornosettimana)
       references giornosettimana(giorno_settimana)
```

```
);

create table al_pc(
   alloggio_id integer not null,
   periodochiusura_id integer not null,
   primary key (alloggio_id, periodochiusura_id),
   foreign key (alloggio_id)
       references alloggio(id),
   foreign key (periodochiusura_id)
       references periodochiusura(id)
);

create table chiusurastraordinaria(
   id serial primary key,
   data_inizio date not null,
   data_fine date not null,
   motivo stringa,
   check (data_inizio < data_fine)
   -- v.inclusione chiusurastraordinaria(id) occorre in
al_cs(chiusura_straordinaria_id)
);

create table al_cs(
   alloggio_id integer not null,
   chiusura_straordinaria_id integer not null,
   primary key (alloggio_id, chiusura_straordinaria_id),
   foreign key (alloggio_id)
       references alloggio(id),
   foreign key (chiusura_straordinaria_id)
       references chiusurastraordinaria(id)
);

create table tipoletto(
   id serial primary key,
   nome_arredo stringa not null,
   formato_letto stringa not null,
   unique(nome_arredo, formato_letto),
   max_capienza intgz not null,
   condivisibile boolean not null
);

create table camera(
```

```sql
   id serial primary key,
   nome stringa not null,
   alloggio_id integer not null,
   unique (nome, alloggio_id),
   foreign key (alloggio_id)
       references alloggio(id)     -- accorpa al_cam
);

create table letto(
   id serial primary key,
   capienza intgz not null,
   camera_id integer not null,
   tipoletto_id integer not null,
   foreign key (camera_id)
       references camera(id),      -- accorpa cam_let
   foreign key (tipoletto_id)
       references tipoletto(id)        -- accorpa let_tip
);

create table prenotazione(
   id serial primary key,
   data_inizio timestamp not null,
   data_fine timestamp not null,
   organiz_id integer not null,
   foreign key (organiz_id)
       references utente(id),       -- accorpa organizzatore
   -- foreign key (organiz)
   --     references visitatore(utente)
   check (data_inizio < data_fine)
   -- v.inclusione prenotazione(id) occorre in visitatore(prenotazione_id)
   -- v.inclusione prenotazione(id) occorre in pren_let(prenotazione_id)
);

create table visitatore(
   utente_id integer not null,
   prenotazione_id integer not null,
   primary key (utente_id, prenotazione_id),
   feed_su_visitatore voto,
   feed_su_ospitante voto,
   foreign key (utente_id)
       references utente(id),
   foreign key (prenotazione_id)
```

```
        references prenotazione(id)
);

alter table prenotazione
   add foreign key (organiz_id, id)
        references visitatore(utente_id, prenotazione_id)
            -- deferrable initially deferred; -- per default il vincolo viene
controllato solo al commit di una futura transazione
            deferrable; -- dovrò scrivere set constraints all deferred in ogni futura
transazione per manipolare i dati

create table prenrifiutata(
   prenotazione_id integer primary key,
   istante_rifiuto timestamp not null,
   motivazione stringa not null,
   foreign key (prenotazione_id)
        references prenotazione(id) -- pr_isa_p
);

create table prenaccettata(
   prenotazione_id integer primary key,
   istante_accettazione timestamp not null,
   foreign key (prenotazione_id)
        references prenotazione(id)  -- pa_isa_p
);

create table pren_let(
   prenotazione_id integer not null,
   letto_id integer not null,
   primary key(prenotazione_id, letto_id),
   foreign key (prenotazione_id)
        references prenotazione(id),
   foreign key (letto_id)
        references letto(id)
);

create table sceltaposto(
   id serial primary key,
   posizione posizione,
   confermato boolean not null,
   utente_id integer not null,
   prenotazione_id integer not null,
```

```sql
   letto_id integer not null,
   foreign key (utente_id)
       references utente(id),  -- accorpa ut_sp
   foreign key (prenotazione_id)
       references prenotazione(id),    -- accorpa pren_sp
   foreign key (letto_id)
       references letto(id),       -- accorpa let_sp
-- [SceltaPosto.coerenza_Prenotazione_Visitatore_Letto_collegati]
-- Per ogni ut: Utente, sp: SceltaPosto, pren: Prenotazione, let: Letto
-- tale che esistono
-- (ut, sp): ut_sp,  ut Utente
-- (pren, sp):pren_sp
-- (let, sp): let_sp
-- deve essere vero che
-- esistono i link (ut, pren): visitatore e (pren, let): pren_let
   foreign key (utente_id, prenotazione_id)
       references visitatore(utente_id, prenotazione_id),
   foreign key (prenotazione_id, letto_id)
       references pren_let(prenotazione_id, letto_id)
);
```